KlintCode Private Agent

Autonomous Code Generation on AMD Hardware

Track 2: Private AI Agents

Project Background & Vision

Data Security Dilemma

Enterprise-grade software development cannot tolerate the risk of leaking proprietary logic, intellectual property, or confidential algorithms to public cloud LLMs.

Context Fragmentation

Traditional coding assistants struggle with complex architectural tasks due to limited context windows, resulting in logical gaps across decoupled files.

Solution: A fully local, air-gapped autonomous agent framework running natively on AMD hardware, capable of end-to-end microservice generation.

System Architecture (M1 - M5 Pipeline)

- **M1 — Initialization:** Ingests business requirements and establishes a dynamic 256K context budget.
- **M2 — Agent Reasoning:** Autonomous planning of domain architecture using local vLLM on ROCm.
- **M3 — Generation:** Asynchronously writes highly decoupled, strictly typed XML domain model
- **M4 — Self-Correction:** Performs syntax checks and enters an autonomous repair loop upon failures.
- **M5 — Compilation:** Translates verified XML models into a compiled Rust microservice library v TeaQL.

AMD Hardware Optimization

- **Hardware Platform:** AMD Radeon RX 7900 XTX
- **Inference Engine:** vLLM (ROCm Optimized)
- **Model:** Qwen3.6-35B-A3B (Quantized)
- **Context Window:** 256,000 tokens

Key Optimizations

Leverages **ROCm PagedAttention** to efficiently manage massive KV caches, enabling the processing of 256K context without OOM (Out Of Memory) failures during long-sequence code generation.

Core Technical Mechanisms

1. Burn-After-Reading Context Management

Dynamically purges validated intermediate code and execution logs from the prompt history, retaining only crucial metadata. This prevents context overflow and maintains inference speed during massive tasks.

2. DAG-Based Schema Validation

Validates inter-model dependencies locally as a Directed Acyclic Graph (DAG) to prevent circular dependencies and isolated endpoints before compilation.

3. Dynamic Temperature Scaling

Progressively increases the LLM temperature parameter during the M4 repair loop, effectively preventing the agent from getting stuck in local optima during syntax corrections.

KlintCode Private Agent

Business Benchmark Validation

Target Scenario

Moving Company Backend Service

Generated from a zero-shot text description outlining vehicles, personnel, finance, and operations.

Output Scale

80+ Enterprise Domain Objects

Equivalent to weeks of human developer effort, fully compiled and verified by the local AMD Radeon GPU in a single session.

Summary & Future Roadmap

- **Practical Value:** Drastically reduces the time and cost of private, enterprise-grade codebase initialization without compromising data sovereignty.
- **Language Expansion:** Planned support for generating Java (Spring Boot) and Go architectures via the TeaQL ecosystem.
- **Dynamic Sharding:** Implementing intelligent memory sharding to support various AMD GPU VRAM specifications for broader developer access.

Contact Information

GitHub: @philipgreat | **Email:** philip_chang@163.com